CMP 519 Software Engineering

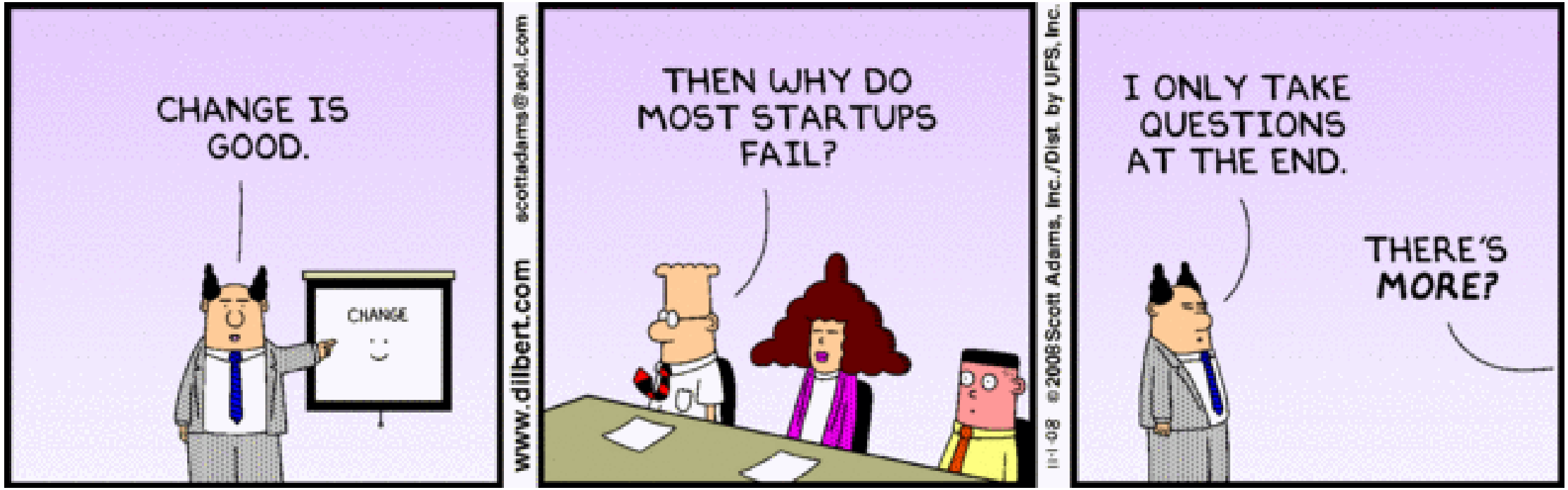
UNIT 07 – Configuration Management and DevOps Tools

Instructor: Dr. Suresh Pokharel

DISCLAIMER

These slides are part of teaching materials for Software Engineering. These slides do not cover all aspect of learning Software Engineering, nor are these be taken as primary source of information. As the core textbooks and reference books for learning the subject has already been specified and provided to the students, students are encouraged to learn from the original sources.

SOFTWARE CONFIGURATION MANAGEMENT (SCM)



These slides are from internet. Link is missing.

SOFTWARE CONFIGURATION MANAGEMENT

Change is inevitable.

AND

Change creates confusion.

BUT,

Confusion is NOT inevitable.

FOUR PRIMARY OBJECTIVES OF SCM

- To **identify** all **items** that collectively **define** the **software configuration**
- To **manage changes** to one or more of these items
- To facilitate the **construction** of different **versions** of an application
- To **ensure** that software **quality** is maintained as the **configuration evolves over time**

Importance of Configuration Management

- Configuration management is a systematic approach that provides:
  - *Traceability*: Tracks all changes made to the software, allowing auditing and rollback when necessary.
  - *Reproducibility*: Ensures that software builds are repeatable across different environments.
  - *Integrity*: Maintains consistency across different environments, preventing configuration drift.
  - *Compliance*: Helps meet industry standards such as ISO 9001, ITIL, and CMMI.
  - *Scalability*: Facilitates the management of complex software architectures with multiple components and dependencies.
- Example: Consider an enterprise software system undergoing frequent updates. Without robust configuration management, inconsistencies between different versions may lead to production failures, increased debugging time, and ultimately loss of customer trust.

WHAT CHANGES? (Configuration Items)

- **Software Code**
  - source code
  - object code
  - Environmental specific configurations
- **Tests**
- **Data including database**
- **Documents**
  - SRS
  - designs
  - project schedules
  - test plans, test results
  - …

KEY CONCEPT: “BASELINE”

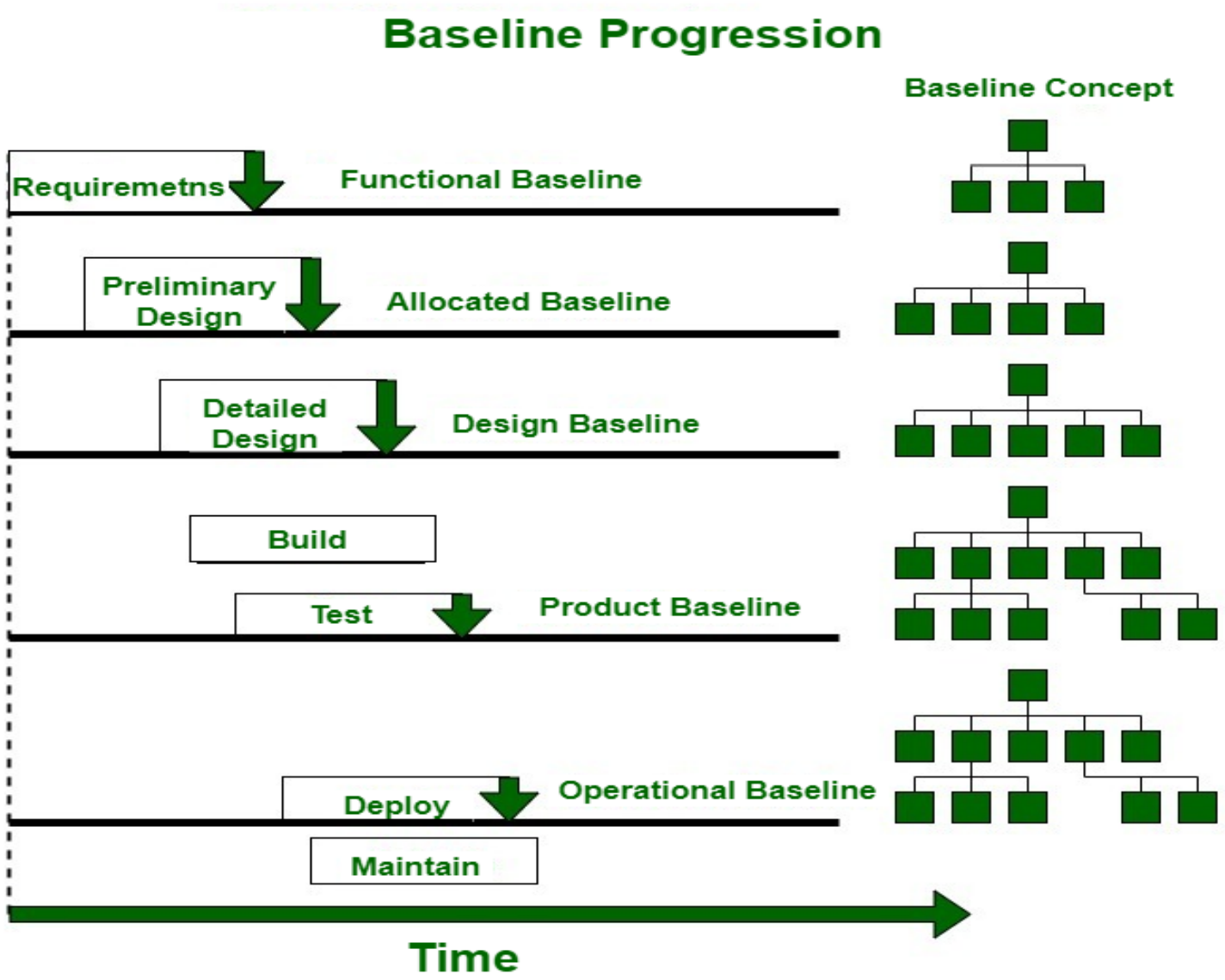
"A specification or product that has been formally reviewed and agreed upon, that there-after serves as the basis for further development, and that can be changed only through formal change control procedures."

IEEE Std 729
Standard Glossary of Software Engineering Terminology

BASELINE

- Establishment of a baseline is a milestone.
- Before that milestone, changes can be made informally.
- Once reviewed and accepted, the product becomes an established baseline. Everyone proceeds using that version, which is located in a central repository.
- Therefore, further change requires formal review, notifications, etc.

BASELINE COMPONENTS



BASELINE – STANDARD CHANGE PROCESS

1. Change Request is made (see next slide for sample)
2. Request is Approved or Denied
3. "Check Out" the item(s)
4. Make Changes
5. Testing or Reviews
    - formal review of design changes
    - Review the Change Order
6. Notify all Dependencies
    - when baseline change will occur
    - what changes were made to baseline
7. "Check In" the new baseline

APPENDIX B
EXAMPLE CHANGE REQUEST FORM

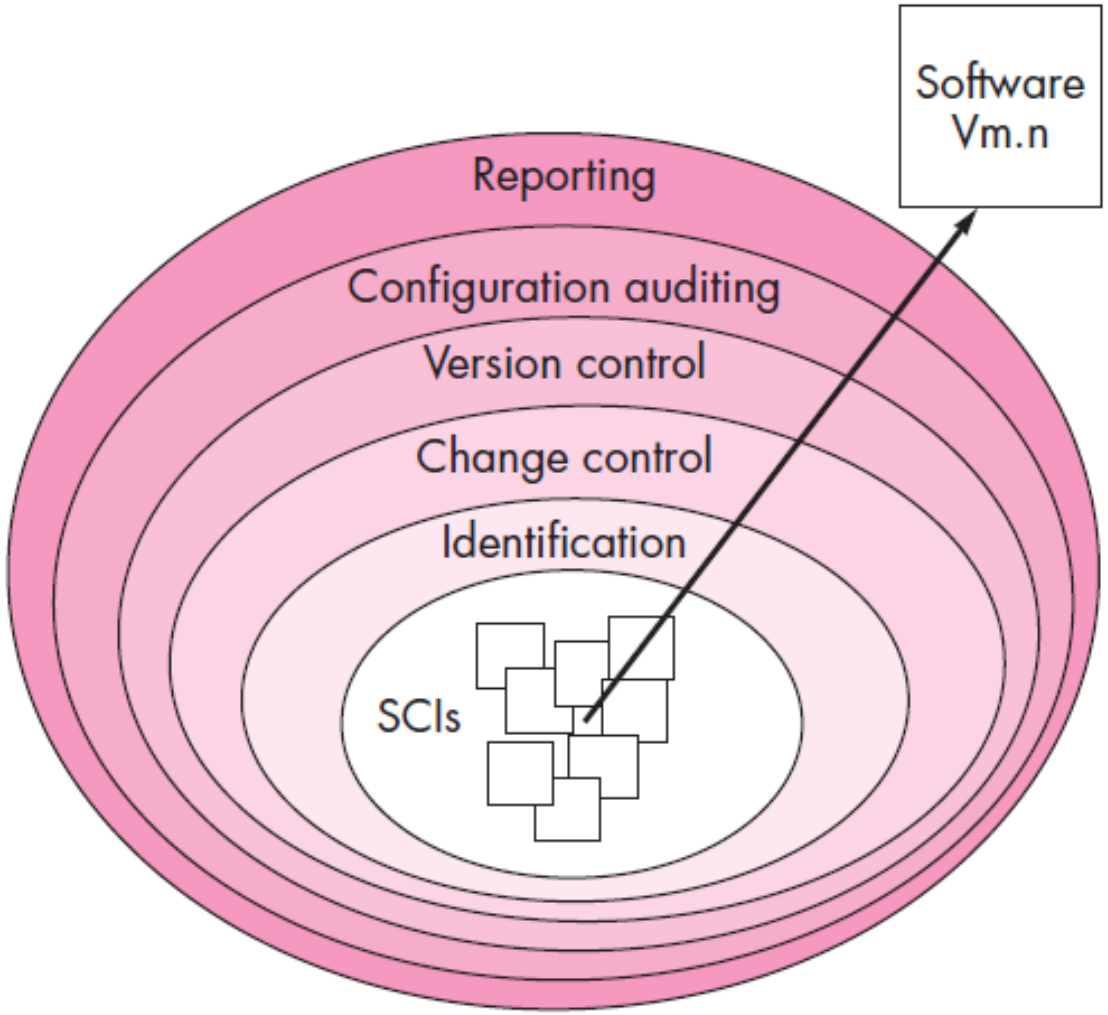
SOFTWARE CHANGE REQUEST	NO.	CLASSIFICATION	PAGE ____ OF ____
TITLE/SUBJECT		SYSTEM/CSCI/CSU	
ORIGINATOR:	DATE SUBMITTED:	DATE NEEDED:	
DESCRIPTION OF CHANGE:			
REASON FOR CHANGE:		TYPE OF CHANGE: ☐ REQUIREMENT ☐ DESIGN ☐ CODE ☐ PROCEDURE ☐ DOCUMENTATION	
IMPACT OF CHANGE:			
DOCUMENTS AFFECTED:	INTERFACES AFFECTED:	COMPONENTS AFFECTED:	
CHANGE IMPACT ASSESSED BY:		ASSESSMENT APPROVED BY:	
CCB DISPOSITION	MEETING DATE:	☐ APPROVED ☐ REJECTED ☐ DEFERRED	
CCB CHAIR: _____			

http://satc.gsfc.nasa.gov/GuideBooks/Image4.jpg

VERSION NUMBERING-EXAMPLE

Item	After Modify	Build 1	Build 2	Release 1…
Test Plan	0.0.0 (original version)			
	0.0.1 -->	0.1.0		
	0.1.1	-->	0.2.0 -->	1.0.0
Module 1	0.0.0 -->	0.1.0 -->	0.2.0 -->	1.0.0
Module 2	0.0.0			
	...			
	0.0.8 -->	0.1.0		
	0.1.1	-->	0.2.0	
	0.2.1			
	0.2.2		-->	1.0.0

LAYERS OF SCM PROCESS



- Identification
  - tracking changes to multiple SCI versions
- Change control
  - authority to approve and prioritize changes
- Version control
  - controlling changes before and after customer release
- Configuration auditing
  - ensure changes are made properly
- Reporting
  - tell others about changes made

SOFTWARE CONFIGURATION IDENTIFICATION

- Effective management of a system requires careful definition of its baseline components, changes to these components need to be defined.
- A system baseline is like a snapshot of the aggregate of system components.
- Updates to this baseline is like frames in a movie strip of the system life cycle.
- The role of software configuration identification in the SCM process is to provide labels for these snapshots and the movie strip.

SOFTWARE CONFIGURATION IDENTIFICATION

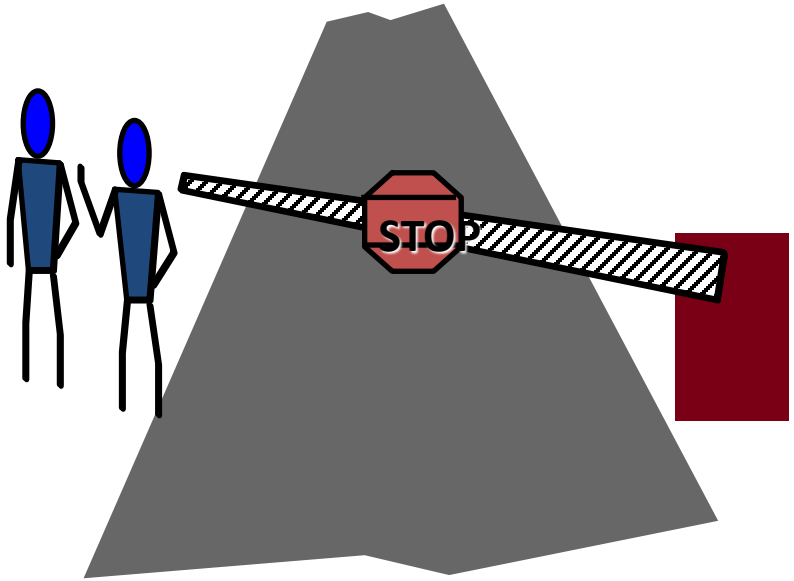
- A baseline can be characterized by two labels, one label for the baseline itself and the other label identifies the update to a particular baseline.
- The most elementary entity in the software configuration identification is the software configuration item (SCI)
- From SCM perspective; a software baseline appears as a set of SCI's.
- The SCI's within a baseline are related to one another via a tree like hierarchy.
- As the software system evolves through its life cycle, the number of branches in this hierarchy generally increases.

SOFTWARE CONFIGURATION IDENTIFICATION

- The first baseline may consist of no more than one SCI.
- Each baseline and each member in the associated family of the updates will exist in one or more forms such as a design document, source code on a disk or executing object code.
- The identification of baselines and updates provides an explicit documentation trail linking all stages of the software life cycle.
- With the aid of this documentation trail, the software developer can assess the integrity of his product, and the software buyer can assess the integrity of the product he is paying for.

SOFTWARE CONFIGURATION CONTROL

- The role of the software configuration control is to provide the administrative mechanism for preparing, evaluating, approving or disapproving all change proposal throughout the system life cycle.
- SCC focuses on managing changes to SCI's (existing or to be developed) in all of their representations.



SOFTWARE CONFIGURATION CONTROL I

need for change is recognized

↓

change request from user

↓

developer evaluates

↓

change report is generated

↓

change control authority decides

↙ ↘

request is queued for action

↓

change request is denied
user is informed

SOFTWARE CONFIGURATION CONTROL II

SOFTWARE CONFIGURATION CONTROL II

↓

assign people to SCIs

↓

check-out SCIs

↓

make the change

↓

review/audit the change

↓

establish a “baseline” for testing

↓

SOFTWARE CONFIGURATION CONTROL III

SOFTWARE CONFIGURATION CONTROL III

↓

perform SQA and testing activities

↓

check-in the changed SCIs

↓

promote SCI for inclusion in next release

↓

rebuild appropriate version

↓

review/audit the change

↓

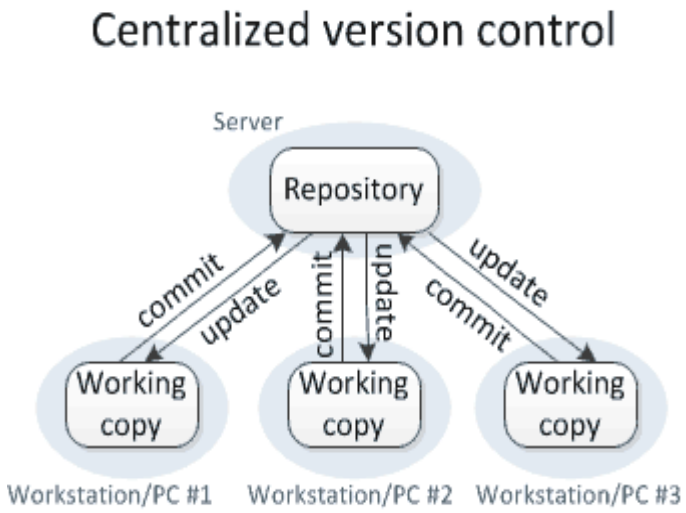
include all changes in release

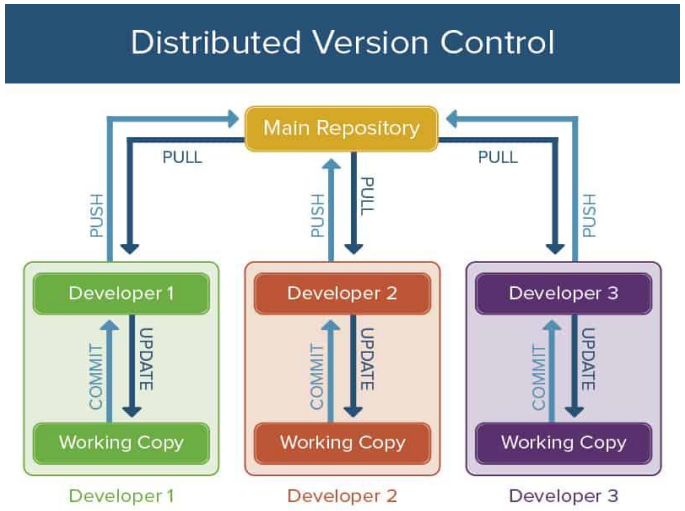
Another Simple Examples (

VERSION CONTROL

- Version control combines procedures and tools to manage different versions of configuration objects that are created during the software process
- A version control system implements or is directly integrated with four major capabilities:
  - a *project database (repository)* that stores all relevant configuration objects
  - a *version management* capability that stores all versions of a configuration object (or enables any version to be constructed using differences from past versions);
  - a *make facility* that enables the software engineer to collect all relevant configuration objects and construct a specific version of the software.
  - an *issues tracking* (also called *bug tracking*) capability that enables the team to record and track the status of all outstanding issues associated with each configuration object.

VERSION CONTROL





- Distributed VCS: Git, Mercurial
- Centralized VCS: CVS, Perforce, SVN

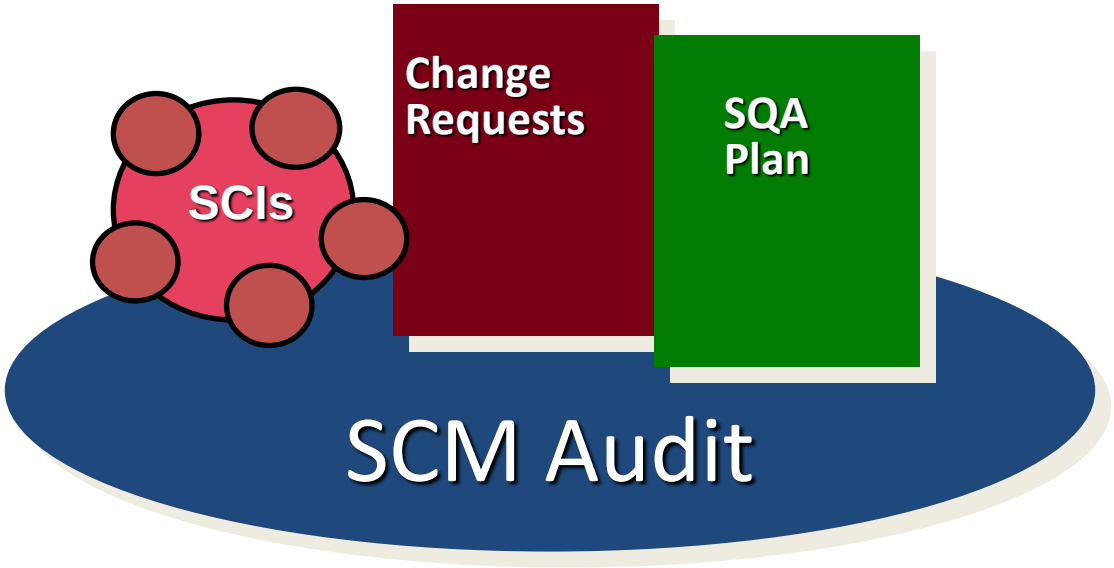
Reference - https://www.oshyn.com/blog/2012/06/version- control-systems-distributed-vs-centralized

VERSION CONTROL

- **Version Control Systems**: Utilizes tools like Git, Subversion (SVN), and Mercurial to manage source code changes.
- **Branching Strategies**:
  - **Git Flow**: Provides structured branching for development, releases, and hotfixes.
  - **Trunk-Based Development**: Encourages frequent integration into the main branch to reduce merge conflicts.
- **Release Management**:
  - **Continuous Deployment**: Automates release cycles to quickly deliver updates.
  - **Feature Toggles**: Enables selective activation of new features.
- **Deployment Strategies**:
  - **Blue-Green Deployments**: Ensures zero-downtime releases.
  - **Canary Releases**: Gradually exposes new updates to subsets of users to monitor impact.

AUDITING

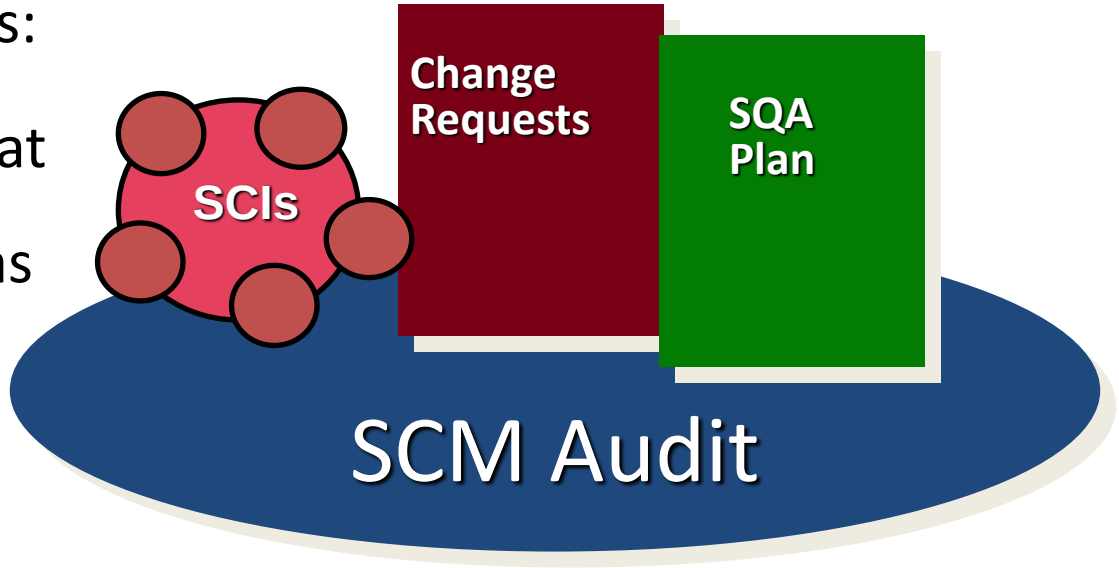
- It provides the mechanism for determining the degree to which the current state of the software system mirrors the software system pictured in baseline and requirements documentation.
- It also provides the mechanism for formally establishing a baseline.



AUDITING

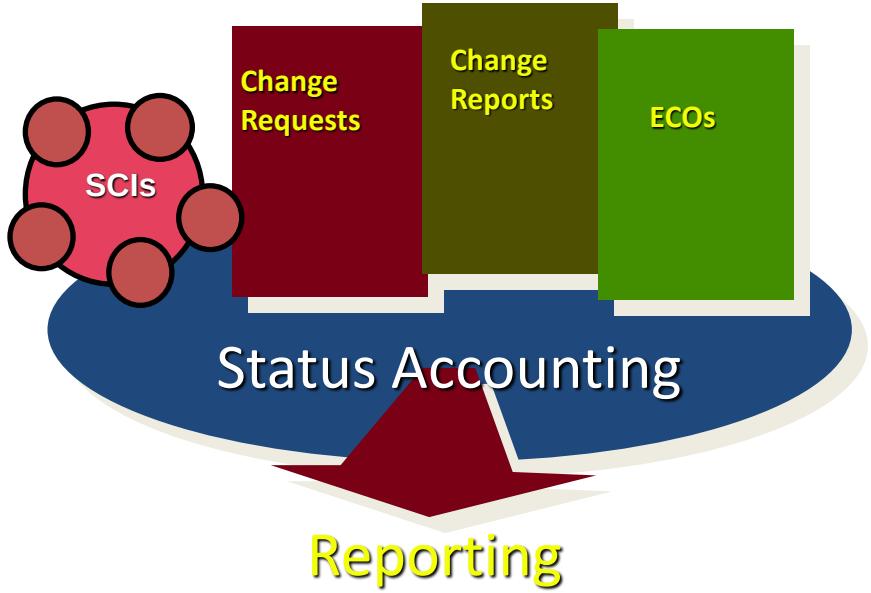
Software configuration auditing serves two purposes:

- Configuration Verification: which ensures that what is intended for each software configuration item as specified in one baseline or update is actually achieve in the succeeding baseline or update.
- Configuration Validation: which ensures that the SCI configuration solves the right problem.



STATUS ACCOUNTING/REPORTING

- It is the administrative tracking and reporting of all software items formally identified and controlled.
- It also involves the maintenance of records to support software configuration auditing.
- It records the activity associated with the other three SCM functions and therefore provides the staffs by which the history of the software system life cycle can be traced.



SOFTWARE CONFIGURATION MANAGEMENT STANDARDS

A number of standards support or include configuration management,[16] including:

- ANSI/EIA-649-1998 National Consensus Standard for Configuration Management
- EIA-649-A 2004 National Consensus Standard for Configuration Management
- ANSI EIA-649-C 2019 Configuration Management Standard
- ISO 10007 Quality management systems – Guidelines for configuration management
- Federal Standard 1037C
- GEIA Standard 836–2002 Configuration Management Data Exchange and Interoperability
- IEEE 829 Standard for Software Test Documentation
- *828-2012 IEEE Standard for Configuration Management in Systems and Software Engineering.*
- MIL-STD-973 Configuration Management (cancelled on 20 September 2000)
- NATO STANAG 4427 Configuration Management in Systems Life Cycle Management including
- NATO ACMP 2000 Policy on Configuration Management
- NATO ACMP 2009 Guidance on Configuration Management
- NATO ACMP 2100 Configuration Management Contractual Requirements
- CMMI CMMI for Development, Version 1.2 Configuration Management
- CMII-100E CMII Standard for Enterprise Configuration Management
- Extended List of Configuration Management & Related Standards
- ITIL Service Asset and Configuration Management
- ISO 20000:1 2011& 2018 Service Management System.
- ECSS-M-ST-40C Rev.1 Configuration and information management

https://en.wikipedia.org/wiki/Configuration_management#Standards

System Building – Build Automation

Build Automation

- Reduces manual effort and errors by automating repetitive build tasks.
- Ensures consistent, repeatable builds across different development environments.
- Uses tools like Maven, Gradle, and Make for dependency management and build scripting.
- **Incremental Builds**: Optimizes compilation by only rebuilding changed components.

System Building – Continuous Integration Practices

Continuous Integration (CI) Practices

- **Automated Code Integration**: Developers integrate code frequently into a shared repository.
- **Automated Testing**: Unit tests, integration tests, and end-to-end tests detect issues early.
- **CI Tools**: Jenkins, GitHub Actions, GitLab CI/CD, and CircleCI facilitate streamlined development.
- **Code Quality Analysis**: Static code analysis tools (SonarQube, ESLint) enhance code maintainability.

Example: A banking application integrates CI pipelines to run security tests on each commit, reducing vulnerabilities in production.

DevOps Tools in Configuration Management

Advanced Use of Jenkins

- Pipeline as Code: Implements CI/CD pipelines using Jenkinsfile scripting.
- Integration with Kubernetes: Automates deployment of microservices-based architectures.
- Security Hardening: Role-based access control (RBAC) and credential encryption.

Containerization with Docker

- Packages applications with dependencies, ensuring consistency from development to production.
- Multi-Stage Builds: Optimizes container images by reducing size and dependencies.
- Docker Compose: Simplifies local development with multi-container environments.

DevOps Tools in Configuration Management

Kubernetes for Container Orchestration

- Automated Scaling: Adjusts resources based on real-time workload.
- Self-Healing: Automatically replaces failed containers.
- Service Mesh Integration: Enhances microservices communication with Istio or Linkerd.

Example: Netflix leverages Kubernetes for global-scale microservices management, optimizing traffic routing and failover mechanisms.

CASE Tools for Configuration Management

- **Git**: A distributed version control system suitable for large-scale collaborative projects.
- **Subversion (SVN)**: A centralized system ideal for strict access control and compliance.
- **Graphical Interfaces**: GitKraken, SourceTree, and TortoiseSVN improve usability for non-expert users.

END OF UNIT 07